## then give me the code of variance of each series

```
import pandas as pd

# Problem 1(b): Sample variance using all available observations per series
returns = pd.read_csv("problem1.csv")

variances = returns.var(axis=0, skipna=True, ddof=1)

print(variances)

# Save the variances to CSV.
variances.to_csv("series_variances.csv", header=["Variance"])
```

## give me code for eigenvalues

```
import numpy as np
import pandas as pd

# Problem 1(c): Eigenvalues of the pairwise correlation matrix
returns = pd.read_csv("problem1.csv")
correlation_matrix = returns.corr(method="pearson", min_periods=2)

eigenvalues = np.linalg.eigvalsh(correlation_matrix.to_numpy())

print("Eigenvalues:")
print(eigenvalues)

# Check positive semi-definiteness with numerical tolerance.
tolerance = 1e-10
print("Positive semi-definite:", np.all(eigenvalues >= -tolerance))
```

## using question2 csv， give me code for var and es

```
import numpy as np
import pandas as pd
from scipy.stats import norm

# Problem 2(a) and 2(b): One-day VaR and ES in percent
prices = (
    pd.read_csv("problem2.csv", parse_dates=["Date"])
```

```
    .sort_values("Date")
    .set_index("Date")
)

# Arithmetic daily returns
returns = prices[["SPY", "XYZ"]].pct_change(fill_method=None).dropna()

# Fit r_XYZ = alpha + beta * r_SPY + epsilon using OLS
market = returns["SPY"].to_numpy()
stock = returns["XYZ"].to_numpy()

X = np.column_stack([np.ones(len(market)), market])
alpha, beta = np.linalg.lstsq(X, stock, rcond=None)[0]
residuals = stock - (alpha + beta * market)

# Estimate market moments and residual variance
market_mean = market.mean()
market_variance = market.var(ddof=1)
residual_variance = np.sum(residuals**2) / (len(stock) - 2)

# Model-implied mean and volatility of XYZ daily returns
mu = alpha + beta * market_mean
sigma = np.sqrt(beta**2 * market_variance + residual_variance)

results = []
for confidence in [0.95, 0.99]:
    z = norm.ppf(confidence)

    # Loss = -return; report VaR and ES in percent
```

## q3 marginal distribution

```
import numpy as np
import pandas as pd
from scipy import stats

# Problem 3(a): Fit marginal distributions for each stock
prices = (
    pd.read_csv("problem3.csv", parse_dates=["Date"])
    .sort_values("Date")
```

```
    .set_index("Date")
)

stocks = [f"S{i}" for i in range(1, 11)]
returns = prices[stocks].pct_change(fill_method=None).dropna()

distributions = {
    "Normal": stats.norm,
    "Student-t": stats.t,
    "Normal Inverse Gaussian": stats.norminvgauss,
}

# Retain fitted distributions for subsequent copula calculations.
fitted_marginals = {}
parameter_rows = []
comparison_rows = []

for stock in stocks:
    data = returns[stock].to_numpy()
    fitted_marginals[stock] = {}

    for name, distribution in distributions.items():
        # Maximum likelihood estimation
        parameters = distribution.fit(data)
        fitted = distribution(*parameters)

        log_densities = fitted.logpdf(data)
        if not np.all(np.isfinite(log_densities)):
```

give me gaussian and t-copula

```
import numpy as np
import pandas as pd
from scipy import stats, optimize
from scipy.linalg import solve_triangular
from scipy.special import gammaln

# Problem 3(b): Fit Gaussian and Student-t copulas
# Run the marginal-fitting code from 3(a) first.
# Enter your chosen marginal for each stock:
```

```
# "Normal", "Student-t", or "Normal Inverse Gaussian"
selected_marginals = {
    "S1": None,
    "S2": None,
    "S3": None,
    "S4": None,
    "S5": None,
    "S6": None,
    "S7": None,
    "S8": None,
    "S9": None,
    "S10": None,
}

if any(choice is None for choice in selected_marginals.values()):
    raise ValueError("Enter your marginal distribution choices above.")

# Transform returns using each selected marginal CDF.
stocks = list(returns.columns)
U = np.column_stack([
    fitted_marginals[s][selected_marginals[s]].cdf(returns[s].to_numpy())
    for s in stocks
])
U = np.clip(U, 1e-10, 1 - 1e-10)
n, d = U.shape

# Parameterize a positive-definite correlation matrix.
```

t-coupula cor

```
# Problem 3(b): Student-t copula correlation matrix
# Run the copula-fitting code above first.

t_copula_correlation = pd.DataFrame(
    R_t,
    index=stocks,
    columns=stocks,
)

print(t_copula_correlation.round(6))
```

```
t_copula_correlation.to_csv("q3_t_copula_correlation.csv")
```

## simulate copula for each position

```
import numpy as np
import pandas as pd
from scipy import stats

# Problem 3(c): Simulate positions using the fitted Student-t copula.
# Run the marginal-fitting and copula-fitting code first.

rng = np.random.default_rng(42)
n_simulations = 100_000
confidence = 0.95

# Generate correlated Student-t draws.
L = np.linalg.cholesky(R_t)
normal_draws = rng.standard_normal((n_simulations, len(stocks))) @ L.T
chi_square_draws = rng.chisquare(nu_t, size=n_simulations)

t_draws = normal_draws / np.sqrt(chi_square_draws[:, None] / nu_t)

# Transform to uniforms, then to returns using the selected marginals.
uniform_draws = np.clip(
    stats.t.cdf(t_draws, df=nu_t),
    1e-10,
    1 - 1e-10,
)

simulated_returns = pd.DataFrame(
    {
        stock: fitted_marginals[stock][selected_marginals[stock]].ppf(
            uniform_draws[:, j]
        )
        for j, stock in enumerate(stocks)
    }
)

# Value positions at the last observed prices.
```

```
holdings = pd.read_csv("problem3_holdings.csv")
```